

where $\text{UNIFY}(l_i, \neg m_j) = \theta$ for example,

we can resolve the two clauses,

$[\text{Animal}(F(x)) \vee \text{Loves}(G(x), x)]$ and $[\neg \text{Loves}(u, v) \vee \neg \text{kills}(u, v)]$

by eliminating the complementary literals $\text{Loves}(G(x), x)$ and $\neg \text{Loves}(u, v)$, with unifier.

$\theta = \{u/G(x), v/x\}$, to produce the resolvent clause.

$[\text{Animal}(F(x)) \vee \neg \text{kills}(G(x), x)]$

- 4) The rule we have just given is the binary resolution rule, because it resolves exactly two literals. The binary resolution rule by itself does not yield a complete inference procedure.
- 5) The full resolution rule resolves subsets of literals in each clause that are unifiable.
- 6) Another approach is to extend factoring. Factoring means the removal of redundant literals in first order case. Propositional factoring reduces two literals to one if they are identical; first order factoring reduces two literals to one if they are unifiable.

Statement proving by resolution :

Resolution proves that $KB \models \alpha$ by proving $KB \wedge \neg \alpha$ unsatisfiable, i.e., by deriving the empty clause. It is a proof by contradiction.

Following are steps taken in resolution proof,

- 1) Convert all problem statements to first order logic.
- 2) Convert first order logic statements into conjunctive normal form.
- 3) Assert the negation of the goal.
- 4) Resolve clauses together until FALSE is derived.

Algorithm always chose to resolve with a clause whose positive literal unifies with the leftmost literal of the 'current' clause. This is exactly what happens in backward chaining.

END... ✍

Unit V

5

Reasoning

5.1 : Inference in First-Order Logic, Propositional vs. First-Order Inference, Unification and First-Order Inference, Forward Chaining, Backward Chaining, Resolution

Q.1 Consider the following facts.

- i) The member of the St. Bridge club are Joe, Sally, Bill and Ellen.
- ii) Joe is married to Sally.
- iii) Bill is Ellen's brother.
- iv) The spouse of every married person in the club is also in the club.
- v) The last meeting of the club was at Joe's house.

- 1) Translate the above sentences into formulas in predicate logic.
- 2) Prove that Ellen is not married.

Ans. : (step I) Predicate logic :

- 1) $\text{MEMBER}(\text{Joe}, \text{ElmStBridgeClub})$
 $\text{MEMBER}(\text{Sally}, \text{ElmStBridgeClub})$
 $\text{MEMBER}(\text{Ellen}, \text{ElmStBridgeClub})$
 $\text{MEMBER}(\text{Bill}, \text{ElmStBridgeClub})$
- 2) $\text{MARRIED}(\text{Joe}, \text{Sally})$
- 3) $\text{BROTHER}(\text{Ellen}, \text{Bill})$
- 4) $\forall x \forall y \text{ MARRIED}(x, y) \wedge \text{MEMBER}(x, \text{ElmStBridgeClub}) \rightarrow \text{MEMBER}(y, \text{ElmStBridgeClub})$
- 5) $\text{LastMeeting}(\text{House}(\text{Joe}))$

• Additional facts :

- 6) $\forall x \forall y \text{ MARRIED}(x,y) \rightarrow \text{BROTHER}(x,y)$
 7) $\forall x \forall y \forall z \text{ MARRIED}(x,y) \rightarrow \neg \text{MARRIED}(x,z)$

(step II) Clause forms :

- 1) $\text{MEMBER}(\text{Joe}, \text{ElmStBridgeClub})$
 $\text{MEMBER}(\text{Sally}, \text{ElmStBridgeClub})$
 $\text{MEMBER}(\text{Ellen}, \text{ElmStBridgeClub})$
 $\text{MEMBER}(\text{Bill}, \text{ElmStBridgeClub})$
 2) $\text{MARRIED}(\text{Joe}, \text{Sally})$
 3) $\text{BROTHER}(\text{Ellen}, \text{Bill})$
 4) $\neg \text{MARRIED}(x,y) \vee \neg \text{MEMBER}(x, \text{ElmStBridgeClub}) \vee \text{MEMBER}(y, \text{ElmStBridgeClub})$
 5) $\text{LastMeeting}(\text{House}(\text{Joe}))$
 6) $\neg \text{MARRIED}(x,y) \vee \neg \text{BROTHER}(x,y)$
 7) $\neg \text{MARRIED}(x,y) \vee \neg \text{MARRIED}(x,z)$

(step III) Resolution :

Resolve with Clause	Subgoal
	$\text{MARRIED}(\text{Ellen}, x)$
6	$\neg \text{BROTHER}(x,y)$
3	Null clause

Hence, proved that Ellen is not married.

Q.2 Consider the facts :

1. Anyone whom Mary loves is a football star.
2. Any student who does not pass does not play.
3. John is a student.
4. Any student who does not study does not pass.
5. Anyone who does not play is not a football star.

Prove using resolution "If John does not study, then, Mary does not love John".

Ans. : (Step I) Covert into Predicate logic :

1. $\forall x (\text{LOVES}(\text{Mary}, x) \rightarrow \text{STAR}(x))$
 2. $\forall x (\text{STUDENT}(x) \wedge \neg \text{PASS}(x) \rightarrow \neg \text{PLAY}(x))$
 3. $\text{STUDENT}(\text{John})$
 4. $\forall x (\text{STUDENT}(x) \wedge \neg \text{STUDY}(x) \rightarrow \neg \text{PASS}(x))$
 5. $\forall x (\neg \text{PLAY}(x) \rightarrow \neg \text{STAR}(x))$

(Step II) Convert into Clause form :

1. $\neg \text{LOVES}(\text{Mary}, x) \vee \text{STAR}(x)$
 2. $\neg \text{STUDENT}(x) \vee \text{PASS}(x) \vee \neg \text{PLAY}(x)$
 3. $\text{STUDENT}(\text{John})$
 4. $\neg \text{STUDENT}(x) \vee \text{STUDY}(x) \vee \neg \text{PASS}(x)$
 5. $\text{PLAY}(x) \vee \neg \text{STAR}(x)$

(Step III) Resolution

Resolve with Clause	Subgoal
	$\neg \text{STUDY}(\text{John}) \vee \text{LOVES}(\text{Mary}, \text{John})$
1	$\neg \text{STUDY}(\text{John}) \vee \text{STAR}(\text{John})$
5	$\neg \text{STUDY}(\text{John}) \vee \text{PLAY}(\text{John})$
4	$\text{PLAY}(\text{John}) \vee \neg \text{PASS}(\text{John}) \vee \neg \text{STUDENT}(\text{John})$
3	$\text{PLAY}(\text{John}) \vee \neg \text{PASS}(\text{John})$
2	$\neg \text{STUDENT}(\text{John})$
3	Null clause

Hence, proved that " If John does not study, then, Mary does not love John".

Q.3 Consider the following sentences.

- 1) Raj likes all kinds of food.
- 2) Apples are food.
- 3) Anything anyone eats and isn't killed by is food.
- 4) Sachin eats peanuts and is still alive.
- 5) Vinod eats everything Sachin eats.

Now attempt following :

Translate the sentences into predicate logic.

Use resolution to answer the question.

Prove that "What food does Vinod eat ?"

Ans. : (Step I) Predicate logic :

- 1) $\forall x \text{ food}(x) \rightarrow \text{likes}(\text{Raj}, x)$
- 2) $\text{food}(\text{Apple})$
- 3) $\forall x \exists y [\text{eats}(y, x) \wedge \neg \text{killed}(y, x) \rightarrow \text{food}(x)]$
- 4) $\text{eats}(\text{Sachin}, \text{peanuts}) \wedge \text{alive}(\text{Sachin}, \text{peanuts})$
- 5) $\forall x \text{ eats}(\text{Sachin}, x) \Rightarrow \text{eats}(\text{Vinod}, x)$

Step (II) Clause form :

- 1) $\neg \text{food}(x) \vee \text{likes}(\text{Raj}, x)$
- 2) $\text{food}(\text{Apple})$
- 3) $\neg \text{eats}(y, x) \vee \text{killed}(y, x) \vee \text{food}(x)$
- 4a) $\text{eats}(\text{Sachin}, \text{peanuts})$
- 4b) $\neg \text{killed}(\text{Sachin}, \text{peanuts})$
- 5) $\neg \text{eats}(\text{Sachin}, x) \vee \text{eats}(\text{Vinod}, x)$

(Step III) Resolution for "What food does Vinod eat?"

Resolve with Clause	Subgoal
	$\neg \text{Eat}(\text{Vinod}, x)$
5	$\neg \text{Eat}(\text{Sachin}, \text{Peanuts})$
4a	Null clause

So, Vinod eats Peanuts.

Q.4 Describe following facts into predicate logic.

1. Every child loves Santa.
2. Everyone who loves Santa loves any reindeer.
3. Rudolph is a reindeer, and Rudolph has a red nose.
4. Anything which has a red nose is weird or is a clown.
5. No reindeer is a clown.
6. Scrooge does not love anything which is weird.

Ans. : Consider the following axioms.

1. Every child loves Santa.
 $\forall x (\text{CHILD}(x) \rightarrow \text{LOVES}(x, \text{Santa}))$
2. Everyone who loves Santa loves any reindeer.

$\forall x (\text{LOVES}(x, \text{Santa}) \rightarrow \forall y (\text{REINDEER}(y) \rightarrow \text{LOVES}(x, y)))$

3. Rudolph is a reindeer, and Rudolph has a red nose.

$\text{REINDEER}(\text{Rudolph}) \wedge \text{REDNOSE}(\text{Rudolph})$

4. Anything which has a red nose is weird or is a clown.

$\forall x (\text{REDNOSE}(x) \rightarrow \text{WEIRD}(x) \vee \text{CLOWN}(x))$

5. No reindeer is a clown.

$\neg \exists x (\text{REINDEER}(x) \wedge \text{CLOWN}(x))$

6. Scrooge does not love anything which is weird.

$\forall x (\text{WEIRD}(x) \rightarrow \neg \text{LOVES}(\text{Scrooge}, x))$

Q.5 Consider the facts :

- 1) Every child loves santa
- 2) Every child loves every candy.
- 3) Anyone who loves some candy is not a nutrition fanatic.
- 4) Anyone who eats any pumpkin is a nutrition fanatic.
- 5) Anyone who buys any pumpkin either carves it or eats it.
- 6) John buys a pumpkin.
- 7) Lifesavers is a candy.

Use resolution and prove : If John is a child, then John carves some pumpkin.

Ans. : (Step I) Predicate logic :

1. $\forall x \forall y (\text{CHILD}(x) \wedge \text{SANTA}(y) \rightarrow \text{LOVES}(x, y))$
2. $\forall x \forall y (\text{CHILD}(x) \wedge \text{CANDY}(y) \rightarrow \text{LOVES}(x, y))$
3. $\forall x ((\exists y (\text{CANDY}(y) \wedge \text{LOVES}(x, y))) \rightarrow \neg \text{FANATIC}(x))$
4. $\forall x ((\exists y (\text{PUMPKIN}(y) \wedge \text{EAT}(x, y))) \rightarrow \text{FANATIC}(x))$
5. $\forall x \forall y (\text{PUMPKIN}(y) \wedge \text{BUY}(x, y) \rightarrow \text{CARVE}(x, y) \vee \text{EAT}(x, y))$
6. $\exists x (\text{PUMPKIN}(x) \wedge \text{BUY}(\text{John}, x))$
7. $\text{CANDY}(\text{Lifesavers})$

(Step II) Clause form :

1. $\neg \text{CHILD}(x) \vee \neg \text{SANTA}(y) \vee \text{LOVES}(x, y)$
2. $\neg \text{CHILD}(x) \vee \neg \text{CANDY}(y) \vee \text{LOVES}(x, y)$

3. $\neg \text{CANDY}(y) \vee \neg \text{LOVES}(x, y) \vee \neg \text{FANATIC}(x)$
4. $\neg \text{PUMPKIN}(y) \vee \neg \text{EAT}(x, y) \vee \text{FANATIC}(x)$
5. $\neg \text{PUMPKIN}(y) \vee \neg \text{BUY}(x, y) \vee \text{CARVE}(x, y) \vee \text{EAT}(x, y)$
6. $\text{PUMPKIN}(x), \quad 6b) \text{BUY}(\text{John}, x)$
7. $\text{CANDY}(\text{Lifesavers})$

(Step III) Resolution :

Resolve with Clause	Subgoal
	$\text{CHILD}(\text{John}) \vee \neg \text{PUMPKIN}(y) \vee \neg \text{CARVE}(\text{John}, y)$
5	$\text{CHILD}(\text{John}) \vee \neg \text{BUY}(\text{John}, y) \vee \text{EAT}(\text{John}, y)$
6b	$\text{CHILD}(\text{John}) \vee \text{EAT}(x, y)$
4	$\text{CHILD}(\text{John}) \vee \neg \text{PUMPKIN}(y) \vee \text{FANATIC}(\text{John})$
6	$\text{CHILD}(\text{John}) \vee \text{FANATIC}(\text{John})$
3	$\text{CHILD}(\text{John}) \vee \neg \text{CANDY}(\text{Lifesavers})$ $\vee \neg \text{LOVES}(\text{John}, \text{Lifesavers})$
7	$\text{CHILD}(\text{John}) \vee \neg \text{LOVES}(\text{John}, \text{Lifesavers})$
2	$\neg \text{CANDY}(\text{Lifesavers})$
7	Null clause

Hence, proved that "If John is not a child, John does not carve pumpkins".

Q.6 Explain the steps of unification in predicate logic. Also discuss the steps of converting predicate logic wffs to clause form.

Ans. : Unification :

- 1) It is the process of finding substitutions for lifted inference rules, which can make different logical expression to look similar (identical).
- 2) Unification is an procedure for determining substitutions needed to make two first order logic expressions match.
- 3) Unification is important component of all first order logic inference algorithms.

- 4) The unification algorithm takes two sentences and returns a unifier for them, if one exists.

The algorithm statement is as follows :-

Unify (p, q) = θ where,

$\text{SUBST}(\theta, p) = \text{SUBST}(\theta, q)$.

For example :

Suppose we have query-whom does Manmohan meets ? i.e first order logic query is Meets (Manmohan, x)

Some answers to this query can be found by searching all the sentences in knowledge base that unify with Meets (Manmohan, x).

Following are results with 4 different sentences that might be in knowledge base.

- i. Unify (Meets (Manmohan, x), Meets (Manmohan, Sonia)) = { x/Sonia }
- ii. Unify (Meets (Manmohan, x), Meets (y, Sharad)) = { x/Sharad, y/Manmohan }

• The general steps for converting first order logic sentences to conjunctive normal form (Clause form)

$$1. (\forall x)(P(x)) \Rightarrow ((\forall y)(P(y) \Rightarrow P(F(x, y))) \wedge \neg(\forall y)(Q(x, y) \Rightarrow P(y))) \text{ (like } A \Rightarrow B \wedge C)$$

2. Eliminate $\Rightarrow$

$$(\forall x)(\neg P(x) \vee ((\forall y)(\neg P(y) \vee P(F(x, y))) \wedge \neg(\forall y)(\neg Q(x, y) \vee P(y)))$$

5. Reduce scope of negation :

$$(\forall x)(\neg P(x) \vee ((\forall y)(\neg P(y) \vee P(F(x, y))) \wedge (\exists y)(Q(x, y) \wedge \neg P(y))))$$

6. Standardize variables :

$$(\forall x)(\neg P(x) \vee ((\forall y)(\neg P(y) \vee P(F(x, y))) \wedge (\exists z)(Q(x, z) \wedge \neg P(z))))$$

$$(\forall x)(\neg P(x) \vee ((\forall y)(\neg P(y) \vee \neg P(y) \vee P(F(x, y))) \wedge (\exists z)(Q(x, z) \wedge \neg P(z))))$$

5. Eliminate existential quantification.

(Skolemization) [The detail procedure is explained next]

$$(\forall x)(\neg P(x) \vee ((\forall y)(\neg P(y) \vee P(F(x, y))) \wedge (Q(x, g(x)) \wedge \neg P(g(x))))$$

7. Drop universal quantification symbols :

$$(\neg P(x) \vee ((\neg P(y) \vee P(F(x, y))) \wedge (Q(x, g(x)) \wedge \neg P(g(x))))))$$

8. Convert to conjunction of disjunctions :

$$(\neg P(x) \vee \neg P(y) \vee P(F(x, y))) \wedge (\neg P(x) \vee Q(x, g(x))) \wedge (\neg P(x) \vee \neg P(g(x)))$$

$$(\neg P(x) \vee \neg P(y) \vee P(F(x, y))) \wedge (\neg P(x) \vee Q(x, g(x))) \wedge (\neg P(x) \vee \neg P(g(x)))P$$

9. Create separate clauses :

$$\neg P(x) \vee \neg P(y) \vee P(F(x, y))$$

$$\neg P(x) \vee Q(x, g(x))$$

$$\neg P(x) \vee \neg P(g(x))$$

10. Standardize variables :

$$\neg P(x) \vee \neg P(y) \vee P(F(x, y))$$

$$\neg P(z) \vee Q(z, g(z))$$

$$\neg P(w) \vee \neg P(g(w))$$

Q.7 Differentiate between declarative and procedural representation of knowledge.

Ans. :

Sr. No.	Procedural knowledge	Declarative knowledge
1.	Follow black box.	Follow white box.
2.	Based on process orientation.	Based on data orientation.
3.	Possible to faster usage.	Based on knowledge in the process of system design.
4.	When we have to achieve in a particular result.	According knowledge format that may be manipulated and analyzed.
5.	Knowing how to do something.	It is knowledge about something.
6.	Simple data type can be used.	Large data type can be used.
7.	Followed in C++ and Cobol.	Followed by SQL.

8.	According programming any simple task in one line code.	According SQL any simple task in one line SQL statement.
9.	Programmer should understand execution.	Programmer should not interact with SQL.
10.	Initially faster but later can be slow.	Initially slower but possible later fast.
11.	Describe a rule.	Apply that rule.
12.	Work on interpreter of language.	Work on data engine with the DBMS.
13.	Example : If Manmohan or Monu is older.	Example : Manmohan is older than monu.

Q.8 Represent following sentences in symbol logic :

i) All students like good teachers. ii) All that glitters is not gold.

iii) Fruits and vegetables are delicious. iv) God helps those who help themselves.

v) Jack and Jill went up the hill

Ans. : Given sentences in symbolic logic

i) All students like good teachers :

$$\forall x, y \text{ student } (x) \wedge (\text{GOOD } (y) \wedge \text{Teacher } (y)) \rightarrow \text{Like } (x, y)$$

ii) All that glitters is not gold.

$$\forall x \text{ Glitters } (x) \rightarrow \sim \text{Gold } (x).$$

iii) Fruits and vegetables are delicious.

$$\forall x \text{ Fruit } (x) \vee \text{vegetable } (x) \rightarrow \text{Delicious } (x)$$

iv) God helps those who help themselves.

$$\forall x \text{ Helpself } (x) \rightarrow \text{Helps } (\text{GOD}, x)$$

v) Jack and Jill went up the hill.

$$\text{Hill } (x) \wedge \text{wentup } (\text{Jack}, x) \wedge \text{wentup } (\text{Jill } x)$$

Q.9 Explain the first order predicate logic (FOL). Explain how well formed formulae (WFF) are defined.

Rewrite the following sentences in FOL

- Coconut is a biscuit.
- Mary is a child who takes coconut.
- John loves children who takes biscuits.
- For a triangle ABC, it is given that the sum of the interior angle is 180 degree.

Ans. : First Order Logic

- It is a expressive language which has well defined syntax and semantics. It uses 3 things to represent the world model :

1) Objects :

- People, houses, numbers, theories, colours, games, wars etc. These are nouns and nounphrases in natural languages.

2) Relations

- Red, round, prime, brother, bigger than, inside, owns, comes between etc. These are associations among objects or properties of objects. Relations can be unary such as green or n-ary such as brother_of.

3) Function :

- Father_of, best_friend, one_more_than etc.
- It is a relation in which there is only one "value" for a given input.

For example : "Two plus five equals seven".

Objects - Two, five, seven.

Relation - equals.

Function - plus.

("Two plus five" is the name for the object that is obtained by applying the function plus to the objects two and five).

For example : "Multitalented King Krishna Ruled Dwarika."

Objects - Krishna, Dwarika.

Relation - Ruled.

Functions - King, Multitalented.

Properties of First Order Logic

- It has ability to represent facts about some or all of the objects in the universe.
- It enables to represent law and rules extracted from real world.
- It is useful language representation in mathematics, philosophy and AI related fields.
- It represents facts in more realistic manner rather than just the true or false statement.
- First order logic makes ontological commitment.

Ontological commitment means what assumptions language makes about the nature of reality. First order logic assumes that real world consists of objects and certain relationship they hold or not among them.

Well formed Formulae (WFF)

- WFFs are also called as sentences. WFF is a predicate applied to the correct number of terms. If P and Q are formulae then following are WFFs,

$$P \rightarrow Q$$

$$P \wedge Q$$

$$P \vee Q$$

$$\sim P$$

- If x is a variable, and P is a formula, then the following are WFFs,

$$\forall x P(x)$$

$$\exists x P(x)$$

$\Rightarrow$ Following sentences in FOL:

- Coconut is a biscuit

Biscuit (Coconut)

- Mary is a child who takes biscuit.

Child (Mary) $\wedge$ Biscuit (coconut) $\rightarrow$ takes (Mary, coconut)

iii) Johns Loves children who takes biscuit.

$$\forall x \text{ child } (x) \wedge \text{takes } (x, \text{biscuit}) \rightarrow \text{loves } (\text{Johns}, X)$$

iv) For triangle ABC, it is given that sum of the interior angle is 180 degree.

$$\begin{aligned} &\text{Triangle } (ABC) \rightarrow \\ &[\text{InteriorAngle } (A) \wedge \text{InteriorAngle } (B) \wedge \text{InteriorAngle } (C)] \wedge \\ &[\text{SUMOF } (\text{InteriorAngle } (A) \wedge \text{InteriorAngle } (B) \wedge \text{InteriorAngle } (C)) \\ &[\wedge \text{Equals } (\text{sum}, 180)] \end{aligned}$$

Q.10 Convert following sentence into predical logic and then prove "Marcus is dead" :

- | | |
|--|---------------------------|
| i) Marcus was a man | ii) Marcus was a pompeian |
| iii) Marcus was born in 40 AD | iv) All men are mortal |
| v) All pompeians died when the volcano erupted in 1979 | |
| vi) No mortal lives longer than 150 yrs | vii) It is now 1991 |
| viii) Alive means not dead | |
| ix) If someone dies, then he is dead all later times. | |

Ans. : Convert following sentences into predicate logic.

i) Marcus was a man.

$\Rightarrow \text{Man } (\text{Marcus}).$

ii) Marcus was a pompeian.

$\Rightarrow \text{Pompeian } (\text{Marcus}).$

iii) Marcus was born in 40 AD.

$\Rightarrow \text{Born } (\text{Marcus}, 40)$

iv) All men are mortal.

$\Rightarrow \forall x : \text{man } (x) \rightarrow \text{mortal } (x).$

v) All pompeians died when the volcano erupted in 1979.

$\Rightarrow \text{Erupted } (\text{Volcano}, 79) \wedge \forall x : [\text{Pompeian } (x) \rightarrow \text{Died } (x, 1979)]$

vi) No mortal lives longer than 150 yrs.

$\Rightarrow \forall x : \forall t_1 : \forall t_2 : \text{mortal } (x) \wedge \text{Born } (x, t_1) \wedge \text{gt } (t_2 - t_1, 150) \rightarrow \text{Dead } (x, t_2)$

vii) It is now 1991

DECODE

A Guide for Engineering Students

$\Rightarrow \text{Now} = 1991$

viii) Alive means not dead.

$\Rightarrow \forall x : \forall t : [\text{Alive } (x, t) \rightarrow \neg \text{Dead } (x, t)]$
 $\wedge [\neg \text{Dead } (x, t) \rightarrow \text{Alive } (x, t)]$

ix) If someone dies, then he is dead at all later times.

$\Rightarrow \forall x : \forall t_1 : \forall t_2 : \text{Died } (x, t_1) \wedge \text{gt } (t_2, t_1) \rightarrow \text{Dead } (x, t_2)$

• This representation says that one is dead in all years after the one in which one died. It ignores the question of whether one is dead in the year in which one died.

$\Rightarrow$ To prove that "Marcus is dead"

$\neg \text{Alive } (\text{Marcus}, \text{now})$

$\uparrow$ (9, substitution)

$\text{Dead } (\text{Marcus}, \text{now})$

$\uparrow$ (10, substitution)

$\text{Died } (\text{Marcus}, t_1) \wedge \text{gt } (\text{now } t_1)$

$\uparrow$ (5, substitution)

$\text{Pompeian } (\text{Marcus}) \wedge \text{gt } (\text{now}, 79)$

$\uparrow$ (2)

$\text{gt } (\text{Now}, 79)$

$\uparrow$ (8, substitute equal)

$\text{gt } (1991, 79)$

$\uparrow$ (compute gt)

nil

Arrive at 'nil' hence the contradiction.

Q.11 Determine whether the following argument is valid : "If a baby is hungry, then baby cries. If the baby is not mad, then he does not cry. If a baby is mad, then his face looks abnormal. Therefore, if a baby is hungry, then his face looks abnormal."

DECODE

A Guide for Engineering Students

Ans. : "If a baby is hungry, then the baby cries. If the baby is not mad, then he does not cry. If a baby is mad, then his face looks abnormal. Therefore, if a baby is hungry, then his face looks abnormal."

• Let, the above given sentences be represented as follows,

p = baby is hungry.

q = baby cries.

r = baby is mad.

s = baby's face looks abnormal.

Above sentences in symbolic representation are as follows,

A) If a baby is hungry, then the baby cries.

$$p \rightarrow q$$

B) If the baby is not mad, then he does not cry.

$$\sim r \rightarrow \sim q$$

C) If a baby is mad, then his face looks abnormal.

$$r \rightarrow s$$

• From the given data, the premise is,

$$p \rightarrow q, \sim r \rightarrow \sim q, r \rightarrow s$$

• And, the conclusion is $p \rightarrow s$

• The argument is represented as follows,

$$(p \rightarrow q, \sim r \rightarrow \sim q, r \rightarrow s) \vdash p \rightarrow s$$

• Consider premises,

$$p \rightarrow q, \underbrace{\sim r \rightarrow \sim q}_{\downarrow}, r \rightarrow s$$

$$\therefore \underbrace{p \rightarrow q, q \rightarrow r}_{\downarrow}, r \rightarrow s$$

$$\therefore \underbrace{p \rightarrow r, r \rightarrow s}_{\downarrow}$$

$$\therefore p \rightarrow s$$

∴ Conclusion is proved.

∴ $(p \rightarrow q, \sim r \rightarrow \sim q, r \rightarrow s) \vdash p \rightarrow s$ holds.

$(\sim r \rightarrow \sim q \equiv q \rightarrow r)$
∴ contrapositive rule)
(∴ chain rule)
(∴ Chain rule)

Q.12 Determine whether the following argument is valid :

"All artists are entertaining people. Some philosophers are mathematicians. Some agents are salesman. Only unentertaining people are salesmen. Therefore, some agents are not philosophers".

Ans. :

p : A person is an Artist

q : A person is entertainer

r : A person is philosopher

s : A person is mathematician

t : A person is agent

u : A person is salesman

From given,

$$\exists x p(x) \rightarrow q(x)$$

$$\exists x r(x) \rightarrow s(x)$$

$$\exists x t(x) \rightarrow u(x)$$

$$\forall x u(x) \rightarrow \sim q(x)$$

$$\forall x \sim q(x) \rightarrow \mu(x)$$

To check validity of

$$\exists x t(x) \rightarrow \sim r(x)$$

From given,

$$\exists x t(x) \rightarrow u(x)$$

$$\therefore \forall x u(x) \rightarrow \sim q(x)$$

$$\exists x t(x) \rightarrow \sim q(x)$$

... Chain rule

• By Venn diagram we can prove that statement :

Some agents are not philosopher

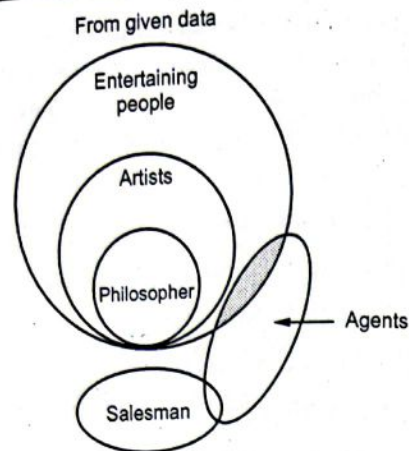


Fig. Q.12.1

Q.13 Write short note on :

Knowledge engineering in first order logic

Ans. : Knowledge Engineering Process in first order logic :

- The process of constructing knowledge base is called as knowledge base engineering. These knowledge base engineering projects vary widely in content, scope and difficulty. A specially designated person called as knowledge engineer will do task of knowledge engineering.

We have Two Types of Knowledge Base

1) General purpose Knowledge base :

Which is intended to support queries about full range of human knowledge. In this, we can expect any kind of query which knowledge base will have to infer.

2) Special purpose Knowledge base :

Which has restricted domain [which can be a certain problem specific domain].

Here expected queries are known in advance.

Steps in Knowledgebase Engineering Process

1) Identify the task :

- The Knowledgebase Engineer must find the range of questions that knowledgebase will support. He should also find the facts that would be available for each specific problem instance.

- Once the task is identified it will determine what knowledge must be represented in order to relate problem instances to answers.
- Note that identifying the task is similar to PEAS designing.

2) Assemble the relevant knowledge :

- This is the process of knowledge acquisition.
- At this stage knowledge is gathered.
- This step helps to understand how the domain actually works.
- This process is carried out by an expert in the underlying domain. (either knowledge base engineer himself or knowledgebase engineer can take expert's help).

3) Decide vocabulary of predicates, functions, and constants :

- This step move towards formal representation of knowledge assembled in step 2.
- Domain-level concepts are translated into logic level names. The process of translation will involve lots of questions about domain like, what is this ?, What it likes ?, What it do ?, How it changes states etc.
- The translated version of problem domain knowledge will contain predicates, functions, constants, terms. All these things together form the vocabulary of problem-domain.
- Vocabulary of problem-domain is called as **Ontology**. The ontology determines what kinds of things exists, but does not determine specific properties and relationship about them.

4) Encode knowledge about the domain :

- In this step knowledge engineer formally list down the axioms for all the vocabulary terms. It is noting of the meaning of the terms, enabling the expert to check the content.
- If some terms are found missing or irrelevant then the mistakes are corrected using step 3 again.
- If additional terms are found they are added to ontology. Invalid terms are removed from ontology. Thus the ontology is updated.

5) Encode a description of the specific problem instance :

- If ontology is well defined encoding the description becomes easy.
- This step involves writing simple atomic sentences about instances of various concepts that are part of ontology.

For example : For a logical agent various problem instances can be supplied by its sensors.

For disembodied knowledge base the instances can be generated by expected input.

6) Fire queries to Inference procedure and get outputs :

- In this step the trials (demos) of querying the knowledge base are conducted.
- The inference procedure is applied on axioms and problem specific facts to get the results.

7) Debug the knowledgebase :

- If step 6 succeeds, its the reward of the first 5 steps.
- But if one is getting unexpected results of queries then knowledge base is debugged for errors.
- There are various reasons why inferencing can fail. The axioms can be missing, axioms can be weak to infer related query, axioms is wrong, are some of the reasons for failure.
- Step 6 and 7 will be carried out repeatedly to make 100 % error free knowledge base.

Q.14. Jacks owns a dog.

Every dog owner is an animal lover.

No animal lover kills an animal.

Either Jack or Curiosity killed the cat, who is named Tuna. By using Resolution prove that.

"Did curiosity kill the cat".

Ans. : FOL representation :

- $\exists x \text{ Dog}(x) \wedge \text{Owns}(\text{Jack}, x)$
- $\forall x (\exists y \text{ Dog}(y) \wedge \text{Owns}(x, y)) \Rightarrow \text{Animallover}(x)$
- $\forall x \text{ Animallover}(x) \Rightarrow \forall y \text{ Animal}(y) \Rightarrow \neg \text{Kills}(x, y)$
- $\text{Kills}(\text{Jack}, \text{Tuna}) \vee \text{Kills}(\text{Curiosity}, \text{Tuna})$
- $\text{Cat}(\text{Tuna})$
- $\forall x \text{ Cat}(x) \Rightarrow \text{Animal}(x)$

CNF representation :

- $\text{Dog}(D)$
 - $\text{Owns}(\text{Jack}, D)$
- $\neg \text{Dog}(y) \vee \neg \text{Owns}(x, y) \vee \text{Animallover}(x)$
- $\neg \text{Animallover}(x) \vee \neg \text{Animal}(y) \vee \neg \text{Kills}(x, y)$
- $\text{Kills}(\text{Jack}, \text{Tuna}) \vee \text{Kills}(\text{Curiosity}, \text{Tuna})$
- $\text{Cat}(\text{Tuna})$
- $\neg \text{Cat}(x) \vee \text{Animal}(x)$

Resolution with refutation using CNF representation :

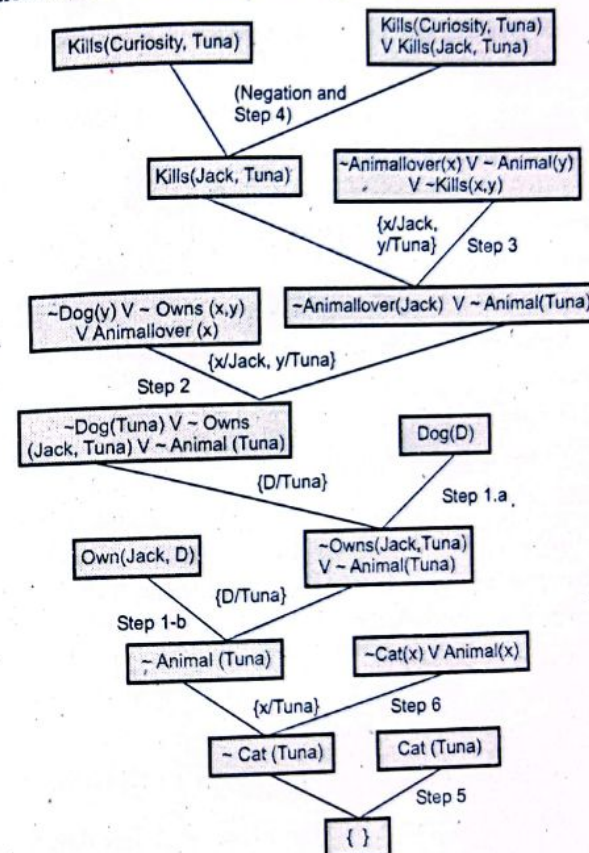


Fig. Q.14.1

- To prove the given task, we assumed the negation of it (i.e.) $\sim$ Kills (Curiosity, Tuna) and using inference rule we derive a contradiction as empty set, which means that the assumption must be false, and Kills (Curiosity, Tuna) is true.

a. $\{x|A, y|B, z|B\}$

b. No unifier (x cannot bind to both A and B)

Q.15 Explain the difference between forward and backward chaining. Under what conditions each will be best to use for a given set of problems.

Ans. : Forward chaining is widely used in deductive databases. Backward chaining is used in linear space and depth first recursive proof searches.

- Forward chaining is sound and complete for first-order definite clauses. Backward chaining has infinite loops hence its incomplete.
- Forward chaining terminates for datalog in finite number of iterations. As backward chaining has repeated subgoals it is inefficient as compared to forward chaining.
- Forward chaining can make some irrelevant fact inference. Backward chaining would never result in irrelevant fact inferences because it makes backward more with goal orientation only.
- Forward chaining is used in production systems where as backward chaining is used in logic programming.

Forward chaining is data driven.

- It is automatic unconscious processing.
- Example - Object reorganization, routine decisions.
- It may do lots of work that is irrelevant to the goal.

Backward chaining is goal driven.

- It is appropriate for problem solving.
- Example : Where are my keys ?,
How do I get into a PhD programme ?
- Complexity of backward chaining can be much less than in size of knowledgebase.

Q.16 Convert the following English statements to statements in first order logic :

- Every boy or girl is a child.
- Every child gets a doll or a train or a lump of coal.
- No boy gets any doll.
- No child who is good gets any lump of coal.
- Jack is a boy.

Ans. : Given english sentences in FOL

i) Every boy or a girl is a child.

$\forall x \text{ Boy}(x) \vee \text{Girl}(x) \rightarrow \text{Child}(x)$

ii) Every child gets a doll or a train or lump of coal.

$\forall x \text{ child}(x) \rightarrow \text{Gets}(x, \text{Doll}) \vee$
 $\text{Gets}(x, \text{train}) \vee$
 $\text{Gets}(x, \text{coal_lump})$

iii) No boy gets any doll.

$\forall x \text{ Boy}(x) \rightarrow \sim (\text{Gets}(x, \text{Doll}))$

iv) No child who is good gets any lump of coal.

$\forall (\text{child } X) \wedge \sim (\text{Good}, X) \rightarrow \sim (\text{Gets}(X, \text{coal_lump}))$

v) Jack is a boy.

Boy (Jack)

Q.17 Determine whether the following argument is valid : If I work whole night on this problem, then I can solve it. If I solve the problem then I will understand the topic. Therefore I will work whole night on this problem, then I will understand the topic."

Ans. : "If I work whole night on this problem, then I can solve it. If I solve the problem, then I will understand the topic. Therefore, I will work whole night on this problem, then I will understand the topic".

Let the above given sentences be represented as follows,

P = I work on a problem.

q = I solve the problem working whole night

r = I understand the topic.

• Above sentences in symbolic representation are as follows.

A) If I work whole night on this problem, then I can solve it working whole night.

$$P \rightarrow q$$

B) If I solve problem, then I will understand the topic.

$$P \rightarrow r$$

From the given data the premise is,

$$P \rightarrow q, \quad q \rightarrow r$$

and the conclusion is,

$$P \rightarrow r$$

The argument is represent as follows,

$$(P \rightarrow q, q \rightarrow r) \vdash P \rightarrow r$$

Consider premises,

$$\underbrace{P \rightarrow q, q \rightarrow r}$$

$$\therefore P \rightarrow r \quad (\because \text{Chain rule})$$

$\therefore$ Conclusion is proved

$$\therefore (P \rightarrow q, q \rightarrow r) \vdash P \rightarrow r \text{ holds.}$$

Q.18 Write short note on :

Forward and backward chaining

Ans. : Forward Chaining :

- 1) Forward chaining is applied to first order logic definite clauses.
- 2) Definite clauses such as, situations $\Rightarrow$ Response are especially useful for systems that make inference in response to newly arrived information. Forward chaining can be more efficient than resolution theorem proving if systems are represented in definite clauses.
- 3) Starting from known fact, it triggers all the rules whose premises are satisfied, and add their conclusions to known facts.
- 4) The process repeats until query is answered (assuming there is only one answer expected) or no new facts are added (that is query remains unanswered).

5) Fact is **not** considered new if it is simply renamed version of existing fact. One sentence is renaming of another sentence if they are identical except for the names of the variable.

6) In the algorithm a situation comes when no more new inferences are possible. At this stage, knowledge base is called as **Fixed Point** of the inference process.

First Order Logic Fixed Point (KnowledgeBases) can include universally quantified atomic sentences.

7) Performance measurement of forward chaining :

- i. It is sound. Every inference is just an application of generalized Modus ponens which is sound.
- ii. It is complete for definite clause knowledge base. It answers query if knowledge base entails it. One can prove completeness of Datalog in simple steps.
- iii. If K- is the maximum arity (number of arguments) of any predicate, P- the number of predicates and n - number of constant symbols then there can be maximum $(P \cdot n)K$ distinct ground facts. After this, algorithm will reach fixed point.

8) Definite clauses with function symbols can generate infinitely many new facts. So extra care should be taken here to avoid infinite knowledge base growth.

9) If the query has no answer the algorithm may fail to terminate in some cases.

Backward Chaining :

- 1) It use generalized Modus ponens backwards to prove query q, and work backwards.
 - It checks if q is known already.
 - Otherwise it proves by backward chaining all premises of a rule concluding q.
- 2) The list of goals can be thought of as a "stack" waiting to be worked on; if all of them can be satisfied, then the current branch of the proof succeeds. The algorithm takes the first goal in the list and finds every clause in the knowledge base whose positive literal, or head, unifies with the goal. Each such clause creates a new recursive call in which the premise, or body, of the clause is added to the goal stack.

- 3) The algorithm uses composition of substitutions. COMPOSE (θ_1, θ_2) is the substitution whose effect is identical to the effect of applying each substitution in turn. That is,

$$\begin{aligned} \text{SUBST}(\text{COMPOSE}(\theta_1, \theta_2), P) \\ = \text{SUBST}(\theta_2, \text{SUBST}(\theta_1, P)). \end{aligned}$$

In the algorithm, the current variable bindings, which are stored in θ , are composed with the bindings resulting from unifying the goal with the clause head, giving a new set of current bindings for the recursive call.

4) Properties of backward chaining :

- 1) It uses depth-first search for proof. It's memory requirement is linear in size of proof.
- 2) It is incomplete due to infinite loops. It should check if new subgoal is already an goal stack.
- 3) It is inefficient due to repeated subgoals (success and failure). It should cache previous results and check for new subgoal.
- 4) We can have two versions of it. Find any solution, OR Find all solutions.
- 5) It is used for logic programming : Prolog

Q.19 Explain the procedure for conversion of FOL to CNF with example.

Ans. : The general steps for converting first order logic sentences to conjunctive normal form

1. $(\forall x)(P(x) \Rightarrow ((\forall y)(P(y) \Rightarrow P(F(x, y))) \wedge \neg(\forall y)(Q(x, y) \Rightarrow P(y))) \text{ (like } A \Rightarrow B \wedge C \text{)})$
2. Eliminate $\Rightarrow$
 $(\forall x)(\neg P(x) \vee ((\forall y)(\neg P(y) \vee P(F(x, y))) \wedge \neg(\forall y)(\neg Q(x, y) \vee P(y)))$
3. Reduce scope of negation :
 $(\forall x)(\neg P(x) \vee ((\forall y)(\neg P(y) \vee P(F(x, y))) \wedge (\exists y)(Q(x, y) \wedge \neg P(y))))$
4. Standardize variables :
 $(\forall x)(\neg P(x) \vee ((\forall y)(\neg P(y) \vee P(F(x, y))) \wedge (\exists z)(Q(x, z) \wedge \neg P(z))))$

$$(\forall x)(\neg P(x) \vee ((\forall y)(\neg P(y) \vee (\neg P(y) \vee P(F(x, y)))) \wedge (\exists z)(Q(x, z) \wedge \neg P(z))))$$

Eliminate existential quantification.

5. (Skolemization) [The detail procedure is explained next]
 $(\forall x)(\neg P(x) \vee ((\forall y)(\neg P(y) \vee P(F(x, y))) \wedge (Q(x, g(x)) \wedge \neg P(g(x))))$

Drop universal quantification symbols :

6. $(\neg P(x) \vee ((\neg P(y) \vee P(F(x, y))) \wedge (Q(x, g(x)) \wedge \neg P(g(x)))))$

Convert to conjunction of disjunctions :

7. $(\neg P(x) \vee \neg P(y) \vee P(F(x, y))) \wedge (\neg P(x) \vee Q(x, g(x)))$
 $\wedge (\neg P(x) \vee \neg P(g(x)))$
 $(\neg P(x) \vee \neg P(y) \vee P(F(x, y))) \wedge (\neg P(x) \vee Q(x, g(x)))$
 $\wedge (\neg P(x) \vee \neg P(g(x)))P$

8. Create separate clauses :

$$\begin{aligned} &\neg P(x) \vee \neg P(y) \vee P(F(x, y)) \\ &\neg P(x) \vee Q(x, g(x)) \\ &\neg P(x) \vee \neg P(g(x)). \end{aligned}$$

9. Standardize variables :

$$\begin{aligned} &\neg P(x) \vee \neg P(y) \vee P(F(x, y)) \\ &\neg P(z) \vee Q(z, g(z)) \\ &\neg P(w) \vee \neg P(g(w)). \end{aligned}$$

Skolemize :

- 1) Skolemization is the process of removing existential quantifier by elimination.
- 2) This methods converts a sentence with existential quantifier into a sentence without existential quantifier such that the first sentence is satisfiable if and only if the second is .
- 3) To eliminate an existential quantifier, replace each occurrence of its variable by a skolem function whose arguments are the variables of universal quantifier whose scope includes the scope of the existential quantifier being eliminated.

- 4) If the existential quantifier being eliminated is not within the scope of any universal quantifier, the skolem function has no arguments that is, it is a constant.

Q.20 What are the limitations in using propositional logic to represent the knowledge base ?

Ans. : Propositional logic has following limitations to represent the knowledge base.

- 1) It has limited expressive power.
- 2) It cannot directly represents properties of individuals or relations between individuals.
- 3) Generalizations, patterns, regularities cannot easily be represented.
- 4) Many rules (axioms) are required to write so as to allow inferencing.

Q.21 Name two standard quantifier.

Ans. : The two standard quantifier are universal quantifiers (represented as $\forall$, which means "For all") and existential quantifier (represented as $\exists$, which means "there exists some object").

They are used for expressing properties of entire collection of objects rather than just a single object.

For example :

- i) $\forall x$ Happy (x) means that "if the universe of discourse is people, then everyone is happy".
- ii) $\exists x$ Happy (x) means that "if the universe of discourse is people, then this means that there is at least one happy person".

Q.22 What is a purpose of unification ?

Ans. : It is used for finding substitutions for inference rules, which can make different logical expression to look identical. It helps to match to logical expressions. Therefore it is used in many algorithm in first order logic.

Q.23 What is ontological commitment (what exists in the world) of first order logic ? Represent the sentence "Brothers are siblings" in first order logic.

Ans. : Ontological commitment means what assumptions language makes about the nature of reality.

Representation of "Brothers are siblings" in first order logic is $\forall x, y [\text{Brother}(x, y) \Rightarrow \text{Siblings}(x, y)]$.

Q.24 Differentiate between propositional versus first order predicate logic.

Ans. : Following are the comparative differences between propositional logic and first order logic

- 1) Propositional logic is less expressive and do not reflect individual object's properties explicitly.

First order logic is more expressive and can represent individual object along with all its properties.

- 2) Propositional logic can not represent relationship among objects whereas first order logic can represent relationship.
- 3) Propositional logic do not consider generalization of objects where as first order logic handles generalization.
- 4) Propositional logic includes sentence letters (A, B, C) and logical connectives, but not quantifier.

First order logic has the same connectives as propositional logic, but it also has variables for individual objects, quantifier, symbols for functions, and symbols for relations.

Q.25 What factors justify whether the reasoning is to be done in forward or backward reasoning ?

Ans. : Following factors justify whether the reasoning is to be done in forward or backward reasoning -

- 1) Which state is more possible to begin with, the start state or goal state ?
- 2) Is there a need to justify the reasoning ?
- 3) What kind of events trigger the problem - solving ?
- 4) In which direction is the branching factor greatest ? One should go in the direction with lower branching factor.

Q.26 Define diagnostic rules with example.

Ans. : Diagnostic rules are used in first order logic for inferencing. The diagnostic rules generate hidden causes from observed effect. They help to deduce hidden facts in the world. For example consider the wumpus world.

The diagnostic rule finding 'pit' is,

"If square is breezy some adjacent square must contain pit", which is written as,

$$\forall s \text{ Breezy}(s) \Rightarrow \exists r, \text{Adjacent}(r,s) \wedge \text{pit}(r)$$

Q.27 Represent the following sentence in predicate form "All the children likes sweets".

Ans. : $\forall x \text{ child}(x) \wedge \text{sweet}(y) \wedge \text{likes}(x,y)$.

Q.28 What is skolemization ?

Ans. : Skolemization is the process of removing existential quantifier by elimination. It converts a sentence with existential quantifier into a sentence without existential quantifier such that the first sentence is satisfiable if and only if the second is.

For eliminating an existential quantifier, each occurrence of its variable is replaced by a skolem function whose argument are the variables of universal quantifier whose scope includes the scope of the existential quantifier.

Q.29 Define the first order definite clause.

Ans. : 1) They are disjunctions of literals of which exactly one is positive.

2) A definite clause is either atomic sentence or is an implication whose antecedants (left hand side clause) is a conjunction of positive literals and consequent (Right hand side clause) is a single positive literal.

For example :

$$\text{Princess}(x) \wedge \text{Beautiful}(x) \Rightarrow \text{GoodHearted}(x)$$

$$\text{Princess}(x)$$

$$\text{Beautiful}(x)$$

Q.30 Write the generalized modus ponens rule.

Ans. : 1) Modus ponens :

If the sentence P and $P \rightarrow Q$ are known to be true, then modus ponens lets us infer Q.

For example : If we have statment, " If it is raining then the ground will be wet" and "It is raining". If P denotes " It is raining" and Q is "The ground is wet" then the first expression becomes $P \rightarrow Q$. Because it is indeed now raining (P is true), our set of sentences becomes,

DECODE®

A Guide for Engineering Students

$$\{ P \rightarrow Q \\ P \}$$

Through an application of modus ponens, the fact that "The ground is wet" (Q) may be added to the set of true expressions.

The generalized modus ponens :

For atomic sentences P_i, P'_i , and q, where there is a substitution Q such that $\text{SUBST}(\theta, P'_i) = \text{SUBST}(\theta, P_i)$,

For all i,

$$\frac{P'_1, P'_2, \dots, P'_n, (P_1 \wedge P_2 \wedge \dots \wedge P_n \Rightarrow q)}{\text{SUBST}(\theta, q)}$$

There are n+1 premises to this rule : - The 'n' atomic sentences p'_i and the one implication. The conclusion is the result applying the substitution θ to the consequent q.

Q.31 Define atomic sentence and complex sentence.

Ans. :

i. An atomic sentence is formed from a predicate symbol followed by a parenthesized list of terms.

For example : Stepsister (Cindrella, Drizella)

ii. Atomic sentences can have complex terms as the arguments.

For example : Married (Father (Cindrella), Mother(Drizella))

iii. Atomic sentences are also called atomic expressions, atoms or propositions.

For example : Equal (plus (two, three), five) is an atomic sentence.

Complex sentences

i. Atomic sentences can be connected to each other to form complex sentence.

Logical connectives, $\wedge, \vee, \neg, \rightarrow$ can be used to connect atomic sentences.

For example :

$$\neg \text{Princess}(\text{Drizella}) \rightarrow \text{Princess}(\text{Cindrella})$$

DECODE®

A Guide for Engineering Students

- ii. (foo (two, two, plus (two, three))) $\rightarrow$ (equal (plus (three, two), five) = true) is a sentence because all its components are sentences, appropriately connected by logical operators.

• Various sentences in first order logic formed using connectives :

- 1) If S is a sentence, then so is its negation, $\neg S$.
- 2) If S_1 and S_2 are sentences, then so is their conjunction, $S_1 \wedge S_2$.
- 3) If S_1 and S_2 are sentences, then so is their disjunction, $S_1 \vee S_2$.
- 4) If S_1 and S_2 are sentences, then so is their implication, $S_1 \rightarrow S_2$.
- 5) If S_1 and S_2 are sentences, then so is their equivalence, $S_1 \equiv S_2$.

Q.32 What is unification ?

Ans. :

- 1) It is the process of finding substitutions for lifted inference rules, which can make different logical expression to look similar (identical).
- 2) Unification is an procedure for determining substitutions needed to make two first order logic expressions match.
- 3) Unification is important component of all first order logic inference algorithms.
- 4) The unification algorithm takes two sentences and returns a unifier for them, if one exists.

Q.33 Differentiate forward chaining and backward chaining.

Ans. :

- 1) Forward chaining is data driven.
 - It is automatic unconscious processing.
 - Example - Object recognition, routine decisions.
 - It may do lots of work that is irrelevant to the goal.
- 2) Backward chaining is goal driven.
 - It is appropriate for problem solving.
 - Example : Where are my keys ?,
How do I get into a PhD programme ?
 - Complexity of backward chaining can be much less than linear in size of knowledgebase.

Q.34 Define meta rules.

Ans. : The rules that determine the conflict resolution strategy are called meta rules. Meta rules define knowledge about how the system will work. For example, meta rules may define that knowledge from Expert 1 is to be trusted more than knowledge from Expert 2. Meta rules are treated by the system like normal rules, but are they are given higher priority.

Q.35 Convert the following into Horn clauses.

$\forall x : \forall y : \text{cat}(x) \vee \text{fish}(y) \rightarrow \text{likes_to_eat}(x,y)$

Ans. : Horn clauses are as follows,

$$\neg \text{cat}(x) \vee \neg \text{fish}(y) \vee \text{likes_to_eat}(x,y)$$

5.2 : Knowledge Representation, Ontological Engineering, Categories and Objects, Events, Mental Objects and Modal Logic, Reasoning Systems for Categories, Reasoning with Default Information

Q.36 Define and discuss ontological engineering.

Ans. : Ontological Engineering

The process of Representing the abstract concepts like actions, time which are related to real world domains is defined as Ontological Engineering.

This process is really tough and lengthy because in real world, objects have many different characteristics with various values. These value do differ over time. In such cases we may need to generalize the objects having similar characteristics.

Upper ontology

The general framework of concepts is called as upper ontology.

It is represented using graphs, in which the general concepts are at the top and the more specific concepts below them.

For example :

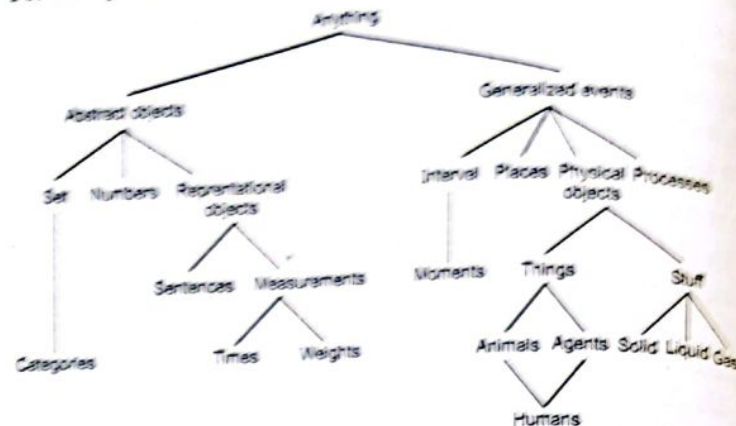


Fig. Q.36.1 The upper ontology of the world each are indicates that the lower concept is a specialization of the upper one

• Knowledge Engineering

A Knowledge Representation (KR) is most fundamentally a surrogate, a substitute for the thing itself, used to enable an entity to determine consequences by thinking rather than acting, i.e., by reasoning about the world rather than taking action in it.

It is a set of ontological commitments, i.e., an answer to the question: In what terms should I think about the world ?

It is fragmentary theory of intelligent reasoning, expressed in terms of three components :

- The representation's fundamental conception of intelligent reasoning ;
- The set of inferences that the representation sanctions ;
- The set of inferences it recommends.

It is a medium for pragmatically efficient computation, i.e., the computational environment in which thinking is accomplished. One contribution to this pragmatic efficiency is supplied by the guidance that a representation provides for organizing information so as to facilitate making the recommended inferences.

It is a medium of human expression, i.e., a language in which we say things about the world.

Our assumption for knowledge representation :

- We are using first order logic for Knowledge Representation.
- Certain aspects of real world are hard to represent using first order logic.
- The major problem is that all most all real world generalizations have exceptions (means special case) or they hold only to a certain degree.

For example :

"Watermelon is dark green" is very useful statement which can be a useful rule in first order logic.

But then in real world there are exceptions to this statements that Watermelon can have other colours like white, lightgreen, cream.

- Exceptions and uncertainty of environment are crucial aspects to be considered in knowledge representation, which we are going to consider later and not in current discussions.

Characteristics of general purpose ontology :

- A general purpose ontology should be applicable in more or less any special purpose domain (with the addition of domain-specific axioms).
- A general purpose ontology should be able to unify and do reasoning for sufficiently large domains and different areas.
- A general purpose ontology should be able to solve problems involving several areas simultaneously.

Q.37 Explain categories and objects along with object measurements.

Ans. : We can do classification of objects in to several categories. Doing categorization helps in knowledge representation. In real world interaction is with the individual object but it really helps to deal with the object at the level of categories. Because of categorization searching would be fast because objects are well organized.

For example : A boy wants to buy a basketball in particular a special basketball such as BB7.

- Categories help to make predictions about objects once they are classified.
- Agent can infer presence of certain objects from perceptual input, and then decide category on the basis of perceived properties of the objects.

For example : From object's property like red, juicy, medium size, oval shape, one can infer that it is a Tomato, which is useful for vegetable salad.

Representation of Categories

We can represent categories in first order logic using either of the two ways

1) **Predicates** : We can use predicate to categorize object.

For example : Member (b, Basketball)

Here Member is predicate and 'b' is a object, categorization under 'Member'

2) **Objects** : We can use object itself to categorized the set of objects.

For example : Basketball (b)

Here 'b' is object categorized as Basketball.

Doing categorization at different levels :

1) If more complex objects are there we can do categorization at different levels.

For example : Subset (Basketballs, Balls)

Here Basketballs is subset of Balls set. That is Basketballs is subcategory (subclass) of Balls.

2) Categories serve to organize and simplify knowledge base through the concept of **Inheritance**.

Inheritance represent objects at specialization levels. A very general category is at the top level and from this level multiple levels can be derived to form inheritance hierarchy for categories.

For example : If we have sentence "All food are edible", then we can have category "food", from 'food' we can create new subset 'fruit'. 'Apple' will belong to category 'fruit'.

From this inheritance hierarchy we can conclude that 'Apple is edible. (as Apple is fruit, fruit is food and food is edible).

3) The **subcategory (subclass)** relation organize categories into taxonomy on taxonomic heirarchy.

Taxonomy helps to provide classification structures in certain domain.

For example : Government departments will have extensive taxonomies of occupations and commercial product.

4) **Category representation in first order logic** : First order logic makes it easy to state facts about categories either by relating objects or by quantifying over their members.

We can do subcategorization till the object is represented in simple format.

Following are some of the first order logic representation for categorization.

- An object is member of a category.

For example : $BBg \in \text{BasketBalls}$.

- A category is a subclass of another category.

For example : $\text{Basketballs} \subset \text{Balls}$.

- All members of a category have some properties.

For example : $x \in \text{Basketballs} \rightarrow \text{Round}(x)$

- Members of a category can be recognized by some properties.

For example : $\text{Orange}(x) \wedge \text{Round}(x) \wedge \text{Diameter}(x) = 9.5 \wedge x \in \text{Balls} \rightarrow x \in \text{BasketBalls}$.

- A category as a whole has some properties.

For example : $\text{Dogs} \in \text{Domesticated Species}$.

5) While specifying categories we should also state **relation between categories that are not subclass of each other**.

For example : If say 'Male and Female are Subclass Animals', then we should also say that 'A male can not be a female'.

We say that two or more categories are disjoint if they do not have any common member.

When we have disjoint subclasses of same category then these subclasses are called as **exhaustive decomposition of the category**.

For example : Disjoint ($\{\text{Animals, Vegetables}\}$)

Exhaustive Decomposition ($\{\text{India, Chinese, Japanese}\}$)

Partition ($\{\text{Males, Females}\}$, Animals)

A disjoint exhaustive decomposition is known as Partition.

6) Categories can be defined by providing necessary and sufficient conditions for membership.

For example : $x \in \text{Bachelors} \Leftrightarrow \text{Unmarried}(x) \wedge x \in \text{Adults} \wedge x \in \text{Males}$.

7) Point to note that : Strict logical definitions for categories are neither always possible nor always necessary.

8) Physical Decomposition :

i) We can group objects into "PartOF" relationship hierarchy.

ii) We use the general "PartOF" relation to say that one thing is part of another.

For example : Kashmir is PartOF India.

iii) The PartOF hierarchy is similar to subset hierarchy.

For example : PartOF (Jammu, Kashmir)

PartOF (Kashmir, India)

PartOF (India, Asia)

PartOF (Asia, Earth)

iv) The PartOF relation is **transitive** and **reflexive** that is,

$\text{PartOF}(x,y) \wedge \text{PartOF}(y,z) \Rightarrow \text{PartOF}(x,z)$.

$\text{PartOF}(x,x)$.

Because of this property referring to our earlier example of 'PartOF' relation, we can conclude that,

PartOF (Jammu, Earth)

v) 'PartOF' relation is useful in representing category of composite objects. Composite objects have structural relations among their parts.

vi) PartPartition Relation : An object is composed of the parts in its partpartition and can be viewed as deriving some properties from those parts.

For example : Mass of composite object is the sum of the masses of the parts.

vii) Another idea to define composite objects is with its definite parts but with no structure.

For example : "The tomatoes in this box weigh 1 kg".

Here the description is about all the tomatoes and not just one tomato.

We use concept called as 'bunch'.

For example : If tomatoes in above collection of 1 kg are, tomato 1, tomato 2, tomato 3, then we have **collection relation**, Bunch Of ({tomato 1, tomato 2, tomato 3})

We can define **BunchOf** in terms of the **PartOF** relation. Each element of S is part of BunchOf (s).

$\forall x \quad x \in S \Rightarrow \text{PartOF}(x, \text{BunchOF}(s))$.

BunchOF (s) is the smallest object satisfying this condition. It means that BunchOF(s) must be part of any object that has all the elements of 's' as parts :-

$\forall y [\forall x \quad x \in s \Rightarrow \text{PartOF}(x,y)]$

PartOF (BunchOF (s),y).

Defining an object as the smallest one, satisfying certain conditions is called as **logical minimization**. This is general technique for deriving axioms.

Measurements

In real-world, object has a important property which represent its quantitative characteristics. Objects have height, mass, cost, properties which can be called as **Measures**. The general quantitative measures are easy to represent. For measuring a object's quantitative feature there are many 'measuring objects' like length, weight etc. These measuring objects can have different names in our language that is length can be measured in feet on meter unit.

- Logically we can combine measuring names as **Unit Function** with a number.

For example : Length (L_1) = Centimeter (100) = Meter (1).

We can convert one unit into another by equating multiples of one unit to another.

- Simple quantitative measures (length, time) are easy to represent. Some qualities like 'beautiful', 'sweet', 'difficult' are really difficult to measure, as we can not assign numbers to them. It is a mistake to measure sweetness on numerical scale, because it is unnecessary.

- In fact in such case we can **order** the measured facts rather than assigning numerical values to them.

For example : Ordering for difficulty can be done as follows.

Aishwarya's question was tougher than Sushmita's and that one, scores less on tough questions.

- i) $q_1 \in \text{Queez} \wedge q_2 \in \text{Queez} \wedge \text{Answer}(\text{Aishwarya}, q_1) \wedge \text{Answer}(\text{Sushmita}, q_2) \Rightarrow \text{Difficulty}(q_1) > \text{Difficulty}(q_2)$
- ii) $(q_1 \in \text{Queez}) \wedge (q_2 \in \text{Queez}) \wedge (\text{Difficulty}(q_1) > \text{Difficulty}(q_2)) \Rightarrow$

Result Score (q_1) < Result Score (q_2).

The sort of relative relationship among measures form the basis for the field of qualitative physics. Qualitative physics is subfield of AI that investigates how to reason about physical systems without plunging into detailed questions and numerical simulations.

The objects and their properties

- Real world is made up of primitive (atomic) objects (particles) and composite objects, build from primitive objects.
- By reasoning at the level of large objects like car, fruits, we can reduce complexity involved in reasoning for large amount of primitive objects.
- The division of objects into distinct objects is called as **individuation**. This division is termed as **Stuff**.

If we divide a stuff into two parts we get two stuff objects.

If we divide a **thing** into two parts we do not get two things.

For example : Water is a stuff, divide it still we get a water (in small amount).

We can represent water is stuff as follows : $x \in \text{Water} \wedge \text{PartOf}(y, x) \Rightarrow y \in \text{Water}$.

Chair is a thing, divide it, we do not get two chairs.

In English the **stuffs** are **mass nouns** and **things** are **count nouns**.

Intrinsic property of the object, is a property, which describes a particle of a object, rather than object as a whole.

When we divide a object, intrinsic property of the divided pieces of object remains the same.

For example :

- Colour, density, boiling point

- Colour of milk is white.

- We divide milk, colour remains same for all divided parts.

- **Extrinsic** property of the object, is a property which describes a object as a whole which is not applicable to its subdivided parts.

For example : Mass, Weight, Length.

If A rope of length 5 meter is divided into two parts, then two parts will not retain five meter length.

- The object that has only intrinsic properties to define is a **substance** or **massnoun**. Stuff is most general substances category, specifying no intrinsic property.

- The object that has only extrinsic properties to define is a thing, or count noun.

- The thing is most general discrete object category, specifying no extrinsic properties.

- All the physical object belong to both substance and thing category. We say that the categories refer to same entities hence categories are **coextensive**.

Q.38 Write a note on ontology of situation calculus. Discuss representational frame problem and ramification problem.

Ans. : The main issue, a knowledgebase based agent deal with is - What is the result of my action ?

A knowledgebase - based agent needs strong reasoning system for this.

The situation (circumstance) for a knowledgebase-based agent will change over the time. Describing all time related changes is really a lengthy task which will take up lot of memory also. Therefore we need to look for situation as such rather than series of situation like situation at time t , and situation at time $t + 1$ and so on. This approach of dealing with situation is called as situation calculus.

- List of assumption

1. We are using first order logic for method representation.
2. Actions are logical terms like Move, Turn (Left)

3. Environment is single-agent. (for multiagent environment we need additional arguments per agent).

Terms in Situation Calculus (Ontology of Situation Calculus)

1. Situations :

- These are logical terms consisting of the initial situation (say S_0) and all the situations that are generated by applying an action to a situation.
- The function Result (a,s) (can be called as D_0) names the situation that results when action 'a' is executed in situation S.

The Q.38.1 diagram depict the result of an action in situation calculus.

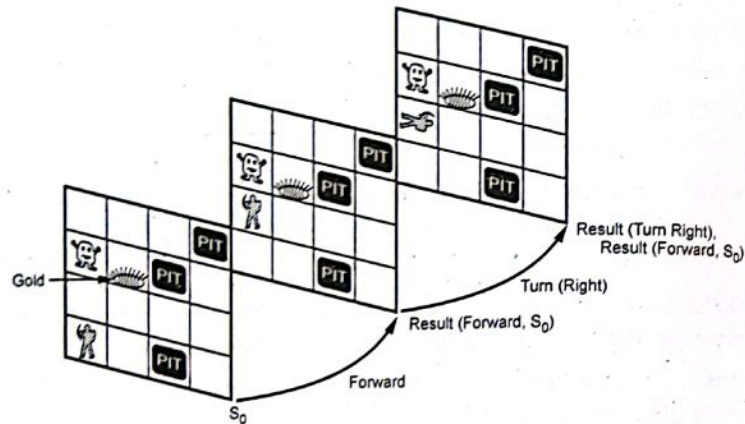


Fig. Q.38.1 In situation calculus, each situation (except S_0) is the result of an action

2. Fluents :

- These are functions and predicates that vary from one situation to the next, such as location of the agent or the status of the agent (like busy, waiting).
- The fluent means flowing or changing across situation.
- By convention, situation is always last argument of the fluent predicate.

For example :

$\neg$ Wears (Crown, S_0).

Cindrella does not wear a crown in situation S_0 .

3. Atemporal predicates :

- These are predicate or functions with single argument.
- They are also called as eternal predicates.

For example :

LeftShoeOf (Cindrella)

4. Projection task :

- It is the task of deducing the outcome of given sequence of actions.
- For finding a effective sequence the knowledgebase-based agent requires good inferencing algorithm.
- The process of finding required action sequence is called as **planning the task**.

For example :

We consider example of Wumus World Agent.

Let agent be at location (1,1) and gold is at location (1,2). Agent's aim is to have gold at location (1,1).

• The fluent predicates

- At (o,x,s) (where, 'o' is a object)
- Holding (o,s)

- As the agent is at location (1,1) the initial knowledgebase will have description,

At (Agent, (1,1), S_0) $\wedge$ At (G_1 , (1,2), S_0)

- We should specify that what is **not** true in S_0 situation.

i. At (o, x, S_0) $\Leftrightarrow [(O = \text{Agent} \wedge x = [1, 1]) \vee (O = G_1 \wedge x = [1, 2])]$

Solving representational frame problem

- Rather than writing down the effects of each action, we will write axioms for, how we get each fluent predicate over the time.

2. The successor-state axioms specify how fluent is evolved from previous state.
3. The format of successor-state axiom is as follows,

$$\text{Action is possible} \Rightarrow [\text{Fluent is true in result state} \Leftrightarrow (\text{Action's effect made it true}) \vee (\text{It was true before and unaffected by the action})]$$
4. This axiom states that fluent will be true if and only if the right hand side holds.

This means that the next state is completely specified from the current state and hence there are no additional frame axioms needed.

For example :

$\text{Poss}(a, s) \Rightarrow$

$(\text{Holding}(g, \text{Result}(a, s)) \Leftrightarrow a = \text{Grab}(g) \vee (\text{Holding}(g, s) \wedge a \neq \text{Release}(g)))$

This axioms means that, agent is holding 'g' after executing an action if the action was a 'grab of g' and the 'grab is possible' or if the agent was already holding 'g' and the action is 'not releasing it'.

5. By writing successor-state axiom, representational frame problem is solved because the total size of the axioms is $O(AE)$ [each of the E effects, of each of the Action is mentioned exactly once], literals instead of $O(AF)$. The literals are spread over F different axioms, so that axioms have average size AE/F .

The ramification problem - Implicit effects

It is a problem associated with implicit effects. Successor state axioms don't consider implicit effects (such as location of things the agent is holding can change). This can be generalized in order to deal with this case as shown below.

$\text{Poss}(a, s) \Rightarrow$

$\text{At}(o, y, \text{Result}(a, s)) \Leftrightarrow$

$(a = \text{Go}(x, y) \wedge (o = \text{Agent} \vee \text{Holding}(o, s))) \vee$

$(\text{At}(o, y, s) \wedge \neg (\exists z y \neq z \wedge a = \text{Go}(y, z) \wedge$

$(o = \text{Agent} \vee \text{Holding}(o, s))))).$

Unique names axiom, Unique action axiom

Technicality : We need to deal with non-identities in the previous formulas. In general, constants in first order logic are not necessarily distinct for the previous axioms to work.

We need to add :

1. For all distinct constants (unique names axiom, often simply assumed by provers), add, $C1 \neq C2$
 2. For all distinct Actions A, B (unique actions axioms), add,
 $A(x_1, \dots, x_m) \neq B(y_1, \dots, y_n)$
 $A(x_1, \dots, x_m) = A(y_1, \dots, y_m) \Leftrightarrow x_1 = y_1 \wedge \dots \wedge x_m = y_m$
- with these added, a theorem prover can prove that the proposed plan.
 $\text{Go}([1,1], [1,2], \text{Grab}(G_1), \text{Go}([1,2], [1,1]))$

achieves the goal.

Solving inferential frame problem

1. There are two techniques to handle inferential frame problem.
2. First technique says that discard situation calculus for writing axioms and use the technique fluent calculus for writing axioms which can reduce considerable amount of step count.
3. Second technique suggests that we should develop better inferencing mechanism, as discussed below.
 - i. For inferencing mechanism we will introduce two new predicates
 $A \Rightarrow \text{Pos EFFECT}(a, F_i)$ - It means that action a causes F_i to become true.
 $B \Rightarrow \text{Neg EFFECT}(a, F_i)$ - It means that a causes F_i to become false.
 - ii. We also need to add new frame axioms.

Format of the axiom will be as follows :

$\text{Poss}(a, s) \Rightarrow$

$F_i(\text{Result}(a, s)) \Leftrightarrow$

$(a = A_1 \vee a = A_2 \dots)$
 Actions making F_i true

$\vee (F_i(s) \wedge a \neq A_3 \wedge a \neq A_4 \dots)$
 Actions making F_i false

For the solution we rewrite the foregoing schema as follows :

$\text{Poss}(a, s) \Rightarrow$

$F_i(\text{Result}(a, s)) \Leftrightarrow \text{PosEffect}(a, F_i)$

$$\vee [F_i(s) \wedge \neg \text{NegEffect}(a, F_i)]$$

$$\text{PossEffect}(A_1, F_i)$$

$$\text{PossEffect}(A_2, F_i)$$

$$\text{NegEffect}(A_3, F_i)$$

$$\text{NegEffect}(A_4, F_i)$$

iii. For efficient inferencing procedure following three things can be done.

1. Once these two predicates are written then index them by their first argument 'a' so that when given an action that occurs at time 't', we can find its effect in $O(1)$ time. (because exact match will be found).
2. These predicates are also indexed by their second argument $F1$ so that when we know $F1$ is an effect of an action we can find axiom for $F1$ in $O(1)$ time.
3. We will represent each situation as a previous situation plus a 'delta' which will tell that something changed from previous state or not. This will eliminate extra axioms, about unchanged situations.
- iv. Thus at each step, we look at the current action, fetch its effects and update the set of true fluents. Each time step will have an average of E of these updates for a total complexity of $O(Et)$ which is reduction in total time. The time has drastically reduced from $O(ft)$ or $O(AEt)$ to $O(Et)$ which has solved inferential frame problem.

$$v. \neg \text{Holding}(O, S_0)$$

- We should specify that G_1 is Gold and location $[1,1]$ and $[1,2]$ are adjacent.

$$\text{Gold}(G_1) \wedge \text{Adjacent}([1,1], [1,2]) \wedge \text{Adjacent}[1,2], [1,1])$$

- Agent will achieve the goal by doing following actions.

- i. Go to $[1,2]$
- ii. Grab the gold
- iii. Return to $[1,1]$

That is,

$$\text{At}(G_1, [1,1], \text{Result}([Go([1,1], [1,2]), \text{Grab}(G_1), Go([1,2][1, 1]), S_0))$$

- Therefore the plan for achieving gold is
 $\exists \text{Seq At}[G_1, [1,1], \text{Result}(\text{Seq}, S_0)]$

Describing Actions in Situation Calculus

Each action can be described by two axioms.

i. Possibility axioms :

That says when it is possible to execute a particular action.

It is of the form :

POSSIBILITY AXIOM : $\text{Preconditions} \Rightarrow \text{Poss}(a,s)$

ii. Effect axiom :

That says what happens when a possible action is executed.

It is of the form :

EFFECT AXIOM : $\text{Poss}(a,s) \Rightarrow \text{changes that result from taking action.}$

For example :

'S' refers to some situation in situations set.

'a' refers to some action in actions set.

'O' refers to some object in objects set including agent.

'g' refers to gold.

'x' and y refers locations.

The possibility axioms, for the wumpus world state, that an agent can go between adjacent locations, grab a piece of gold in the current location and release some gold that it is holding, are as follows,

$$\text{At}(\text{Agent}, x, s) \wedge \text{Adjacent}(x, y) \Rightarrow \text{Poss}(\text{Go}(x, y), s)$$

$$\text{Gold}(g) \wedge \text{At}(\text{Agent}, x, s) \wedge \text{At}(g, x, s) \Rightarrow \text{Poss}(\text{Grab}(g), s)$$

$$\text{Holding}(g, s) \Rightarrow \text{Poss}(\text{Release}(g), s)$$

The effect axioms, with respect to above possibility axioms can be given as follows.

$$\Rightarrow \text{Poss}(\text{Go}(x, y), s) \Rightarrow \text{At}(\text{Agent}, y, \text{Result}(\text{Go}(x, y), s))$$

i.e Going from x to y results in being at y.

$$\Rightarrow \text{Poss}(\text{Grab}(g), s) \Rightarrow \text{Holding}(g, \text{result}(\text{Grab}(g), s))$$

i.e Grabbing the gold results in holding the gold.

$$\Rightarrow \text{Poss}(\text{Release}(g), s) \Rightarrow \neg \text{Holding}(g, \text{result}(\text{Release}(g), s))$$

i.e Realising gold results in not holding gold.

Problems in effect axioms :

1. Effect axioms say that, 'what changes' but do not say anything about 'what stays the same'.
2. The problem related to representation of all the things that stay the same is called as **frame problem**.
3. We should note that in real world, almost everything remains the same almost all the time. Each action affects only a tiny fraction of all fluents.
4. We can solve frame problem using frame axioms. These frame axioms will say that what stays the same.

For example : Agent's movements leave other objects stationary unless they are held. $At(o, x, s) \wedge (0 \neq Agent) \wedge \neg Holding(0, S) \Rightarrow$

$At(o, x, Result(Go(y, z), S))$

5. If there are F fluent predicates and A actions, then we will need O (AF) frame axioms. This is a serious aspect because it will take huge memory to store as well as long time for referencing when required.
6. If action has at most E effects, where E is typically much less than F, then we should be able to represent much smaller knowledge base of size O(AE).

This problem of size of knowledge base is termed as **representational frame problem**.

The **Representational frame problem** addresses the memory requirement issue. It says that size of knowledge base should depend on number of actions and effects and not fluents.

7. **The inferential frame problem** : If we want to project the result of a t-step sequence of known actions, we should be able to do so in time O(Et) rather than O(Ft) or O(AEt). **The inferential frame problem address the time requirement issue.**

Q.39 Discuss time and even calculus.

Ans. : Similar to situation calculus we can have Event Calculus for formalising description about the problem domain.

Situation calculus works when there is a single agent performing instantaneous, discrete actions.

Event calculus is better formalization when actions have duration and actions can overlap each other.

Properties of Event Calculus

1. It is based on points in time rather than various situations.
2. In its context the word action and event are same.
3. Fluent is true at the given point of time. If the fluent was initiated by an event at some time in the past, and was not terminated by an intervening event.
4. There are different relations 'and' for specifying axioms.
5. **Initiates Relation** - Initiates (e, f, t) means that, the occurrence of event 'e' at time 't' causes fluent 'f' to become true.
6. **Terminates Relation** - Terminates (w, f, t) means that, f ceases to be true.
7. Happens (e, t) relation states that event 'e' happens at time 't'.
8. Clipped (f, t, t₂) relation states that 'f' is terminated by some event sometime between 't' and 't₂'.
9. Formal specification of axioms, [The EVENT CALCULUS AXIOM] :

$T(f, t_2) \Leftrightarrow \exists e, t \text{ Happens}(e, t) \wedge \text{Initiates}(e, f, t) \wedge (t < t_2) \wedge \neg \text{Clipped}(f, t, t_2)$

$\text{Clipped}(f, t, t_2) \Leftrightarrow \exists e, t, \text{ Happens}(e, t_1) \wedge \text{Terminates}(e, f, t_1) \wedge (t < t_1) \wedge (t_1 < t_2).$

For example :

Happens (TurnsOn (Lightswitch, 2:25))

Means that lightswitch was turned on at exactly 2:25.

10. There are various extensions of event calculus which consider other aspects of problem domain like-event with duration, continuously changing events, indirect effects, causal constraints, etc.

Generalized Events

A physical universe (problem environment) has both a spatial and temporal dimension. We can represent this physical universe with four dimensions (that is using four aspects). Four dimensional continuous space-time is an adequate representation for commonsense reasoning purpose.

For example :

The wumpus world has its spatial components, as, two-dimensional grid and has discrete time. So it is three dimensional description.

Generally, practical problem world is described with three spatial components and one temporal, where all four components are continuous.

- A event composed of multidimensional space-time universe is called as **generalized event**. Generalized event generalizes actions, locations, time, fluents, physical objects.

For example :

Kargil War Event took place at various points in space time. We can break down Kargil War event in to subevents like,

SubEvent (BattleOfSiachin, KargilWar)

Similarly Kargilwar is subevent of 20th century

SubEvent (KargilWar, TwentiethCentury)

Here, 20th century is an interval of time.

- To denote physical area bounded by border we can use **place** predicate. We can use **In** (like PartOf) predicate to tell that someplace is in another place.

For example :

Place (India)

In (TajMahal, India)

- To denote a place which enclose the event e we can use predicate location (e).

For example : In (location (WorldCup 2011 AsiaSubContinent).

- We can catagorize events based on various aspects like time of event, effect of action, action of other agents etc.

For example :

'GO' can be a category to tell various 'GO' actions like,

GO(x,y), GOTO(y), GOFrom(x)

Here, $GO(x,y) \subseteq GOTO(y)$

$GO(x,y) \subseteq GOFrom(x)$

- We have seen that physical object can be represented as generalized events. We should note following points so that there is no confusion while modeling.

1. A logical object denotes a different object at different time.

For example : Priminister (India)

The objects denoted by above logical term changes overtime like Pandit J.Neharu to ManMohan Singh. But at a time it has exactly one value in the model.

2. There should be clear distinction between a single, as a whole physical object and collections of atoms.

For example : Priminister ManMohan Singh is not equal to collection of all Priminister of India.

Intervals

Intervals are chunks of space-time that include all of space between two time points.

- i. The function period (e) denotes the smallest interval enclosing event e.
- ii. Duration (i) is the length of time occupied by an interval.
- iii. Intervals can be categorized as
 - a. **Moments Intervals** : Where duration is zero seconds.
 - b. **Extended intervals** : Where duration has some length.
- iv. The **extended intervals** have **start point** and **end point**. The start point is earliest moment in interval, where as, the end point is latest moment in interval. The 'Duration' gives the difference between start point and end point.
- v. We can use second, minutes, hour, date, etc for representing interval's numeric values.
- vi. Predicates in interval :

Meet (i,j) $\Leftrightarrow$ Time (End(i)) = Time (Start(j))

Before (i,j) $\Leftrightarrow$ Time (End(i)) < Time (start (j))

After (j,i) $\Leftrightarrow$ Before (i,j).

During (i,j) $\Leftrightarrow$ Time (start(j)) $\leq$ Time (start(i)) $\wedge$ Time (End(i)) $\leq$ Time (End(j)).

Overlap (i,j) $\Leftrightarrow \exists k$ During (k, i) $\wedge$ During (k,j).

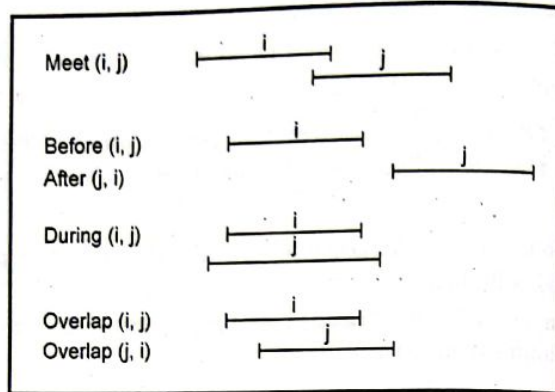


Fig. Q.39.1 Predicates on time intervals

Processes

An event containing many subevents, each one taking some part of interval of the event, is called as **Process**, or **liquid event**. The process event naturally possess property of being continuous change.

For example :

Ram is going from Ayodhya to Lanka.

[Here there is continuous change in Ram's journey procedure].

We can have liquid events which describe processes of continuous non-change, which are called as **states**.

For example : Ram is in Lanka.

Q.40 What is fluent calculus ?

Ans. : We can form more complex states and events by combining primitives which is termed as **fluent calculus**. The combinations can be done with functions, like Both (e_1, e_2) (where e_1, e_2 are events).

Fluent calculus makes use of **infix notation** for representing combination, as shown below :

$e_1 \text{ O } e_2$

"O" is function which is commutative, associative like logical conjunction.

For example :

$\exists p, i (P \in \text{People}) \wedge T [(\text{Watch movie } (P) \text{ O } \text{EatPopcorn } (P)), i]$

Here i, is time interval.

T, is time function.

Q.41 Write a note on

- (i) Mental events and mental objects
- (ii) Theory of beliefs
- (iii) Association of knowledge, belief, time, actions

Ans. :

(i) Mental events and mental objects :

AI agent should be able to deduce beliefs (facts which are true) from current knowledge it has. Agent should possess knowledge about beliefs or about deducing the beliefs.

For single-agent environment, knowledge about one's own knowledge and the reasoning process is useful for controlling inference.

For example :

Agent want to buy a pen. As per agent's knowledge agent will reach to a stationary shop to purchase pen.

In multi-agent environment, the knowledge and reasoning process becomes important to reason about mental state of the other agent. To buy a 'pen' of certain brand (where the pen is ?) agent should enquire it to shop owner (another agent). This process will happen only if agent has the knowledge that "for various unknown brands I should ask the shopkeeper (because he has knowledge about it)."

One's own knowledge help to construct future plans that helps to take relevant action.

To make belief, knowledge and reasoning theory to work, we need to model the mental objects that are in someone's head (that is something's knowledge base) and the mental processes that manipulate them.

Note that we are not interested in how much time reasoning takes or which neurons will fire when live objects (animals) is faced with

particular visual-audio stimulus. What we need is the ultimate solution (inference) and not how we got it.

(II) Theory of beliefs :

1. The relationship between agent and mental - objects is called as **propositional attitudes**.

2.

- We can have relationships like Believes, knows, wants, etc.

For example : Belives (Anil, Bats_Well (sachin))

This means that 'Anil believes that Sachin bats well'.

- Believes is a relation between an agent and propositional fluent. The second argument of Believes relation should be a object, fluent and not a sentence.

- For writing relationships we may have to convert propositions into objects or fluents. The process of converting propositions into objects/fluents is called as **reification**.

For example : Sachin bats well.

Bats (Sachin, well) is a sentence. For Believes relationship we need to do reification of 'Bats' as Bats_well (Sachin) this is a fluent.

3. While writing relationship we may find that same term is existing with different values. The property of being able to **substitute** a term freely for an equal term is called **referential transparency**. In first order logic every relation is referentially transparent.

For example : Dances (Shammi) and Dances (Shamsher Raj)

Here Shammi and Shamsher Raj are different descriptions of same object, and they are referentially transparent i.e Shammi = Shamsher Raj.

4.

- In some cases we may want to write Belives relations in which second argument is referentially opaque that **one cannot substitute an equal term** for the second argument without changing the meaning.

- For developing refer opaque terms we have two methods.

i) **Modal Logic** : A modal in which the propositional attitudes are modal operators which are necessarily referentially opaque.

ii) **Use of Syntactic Theory** : Here mental objects are represented using strings (set of characters). The Unique string axiom are written for strings which states that two strings are equal if and only if they consist of identical characters.

By using Unique string axioms then,

Shammi $\neq$ Shamsher Raj.

We need to define various first order logic functions that can map strings to objects. We can use functions like Den (for denoting), Name (for naming) for objects.

For example :

Shammi and Shamsher Raj are objects both referred by constant symbols 'Dancer'. Then we can have representation as

Den (Shammi) = Dancer $\wedge$

Den (Shamsher Raj) = Dancer.

5. Defining inference Rule for Logical agent :

- Logical agent can use Modus Ponens for inferencing. That is,

If it believes P and believes $p \Rightarrow q$,

then it will also believe q.

- The axiom for referencing is, logical Agent $(a) \wedge$ Believes $(a,p) \wedge$ Believes $(a, \text{concat} ("p" \Rightarrow "q")) \Rightarrow$ Believes (q) .

(concat is function on strings that concatenates their elements)

If the string is underlined and unquoted in axiom, it is a variable for which we can substitute value else it is a literal which should be directly used.

6. Logical Omniscience :

It means that logical agent can deduce any consequence of its beliefs infallibly.

For example : Following rule says that if a logical agent believes something, then it believes that it believes (its own knowledge about beliefs).

Logical Agent $(a) \wedge$ Believes (a,p)

Believes $(a, \text{"Believes (Name(a), p)"})$.

(iii) Association of knowledge, belief, time, actions**I. Knowledge and belief :**

1. Knowledge is justified true belief.
2. For representing knowledge we can have relations like,
 - Knows (a, p) $\rightarrow$ [agent 'a' know that proposition P is true.]
 - KnowsWhether (a, p) $\Leftrightarrow$ Knows (a, p) $\vee$ knows (a, $\neg$ p) $\rightarrow$ [whether agent is knowing P.]
 - KnowsWhat (a, p) $\rightarrow$ [tell that agent is knowing what].

For example :

- i) Knows (Anil, Is (Sachin, Goodbatsman))
 - ii) KnowsWhether (Anil, Is (Sachin, Good-batsman)) $\Leftrightarrow$ Knows (Anil, Is (Sachin, Goodbatsman)) $\vee$ knows (Anil, $\neg$ Is (Sachin, Goodbatsman)).
 - iii) KnowsWhat (Anil, ScoreOf (y)) $\Leftrightarrow$

$$\exists x \text{ knows (Anil, "x = ScoreOf (y)") } \wedge x \in \text{DigitStrings.}$$
- KnowsWhat can also be represented as 3 argument relation - agent, term and catagory of term.
- For example :
- i) KnowsWhat (Agent, "ScoreOf (Sachin)", Digitstrings)
 - ii) KnowsWhat (Agent, "Capital (India)", ProperNames)

II. Knowledge, time, action :

1. Agent deals with beliefs that can change overtime.

For example :

T (Believes (Dipu, "IceRainIn (Shimala)"), (InMonth (December)).
2. Actions can have knowledge preconditions and knowledge effects.

For, dialing a phone number of Anil agent should know phone number, at time 't'.

For example :

Initiates [Lookup (a, "PhoneNumber (Anil)"),

KnowsWhat (a, "PhoneNumber (Anil)", Digitstring)t]

3. Runtime variables are used to represent unknown design time quantities which will get values at the instance time. These are useful for plans that, can be use to gather information and use it.

For example : The plan to look up Anil's number and then dial it can be represented as,

[Lookup (Agent, "PhoneNumber(Anil)", n, Dial (n))

'n' is runtime variable.

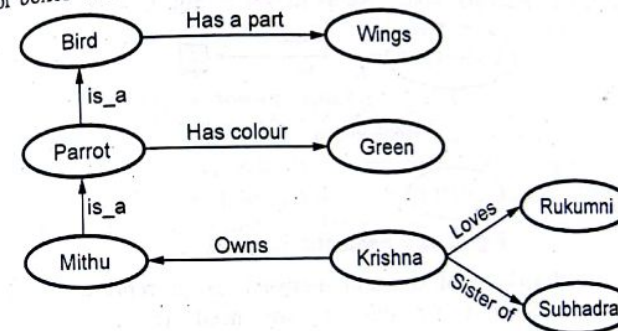
Q.42 Explain organization of categories and their reasoning systems.**Ans. : Organizing Categories**

For organizing categories there are two types of systems

- i. Semantic Networks
- ii. Description Logic.

Semantic Networks

1. It provides graphical aids for visualizing a knowledgebase and efficient algorithms for inferring properties of an object on the basis of its category membership.
2. Semantic Network is graph which has nodes representing objects and their categories, and arc representing relationships between objects.
3. A typical graphical notation displays object or category names in oval or boxes and connects them with the labeled arcs.

**Fig. Q.42.1 Semantic network**

4. Inheritance association (Is a relationship) can be described using semantic network. It is useful for inferencing a query on the object. It can happen that a particular object is subset of more than one catagory (multiple inheritance association) then inferencing algorithm might find two or more conflicting values answering the query for that object.

For example :

Whale is a Mammals.

Ram is a Person.

5. Inverselinks is one common inferencing technique used in semantic network.

For example :

$\forall P, B \text{ HasBrother}(P, B) \Leftrightarrow \text{BrotherOf}(B, P)$

Here relationship is two way

6. Semantic network provide direct indexing for objects, catagories and the links between them.

7. Semantic Network can represent default values for catagories.

For example :

If there is category WildAnimals with following descriptions,
Category (WildAnimals)

Then, we specify default value for legs of WildAnimals,

$\forall x x \in \text{WildAnimals} \Rightarrow \text{Legs}(x, 4)$

[It means that if any animal who is WildAnimals will have four legs.
We need not specify again].

Tiger $\in$ Wild Animals [It means that tiger has four legs].

The semantics Network would like as shown in Fig. Q.42.2,

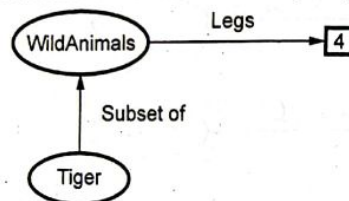


Fig. Q.42.2 Semantic network

8. The major drawback of semantic network is, it represents binary relationships. Solution for this is, we need to convert n-ary associations into binary by reifying the propositions as events.
9. If somethings are unable to expressed in semantic network then semantic network system provides procedural routines attached with the network of objects. Procedure attachment is a technique whereby a query about a certain relation result in a call to a special procedure designed for that relation.

Description Logic

1. These are the notations that are designed to describe definitions and properties of categories.
2. Description logic systems formalize information represented by the semantic network.
3. The inference techniques used in description logic are,

- i. Subsumption : Checking if one catagory is a subset of another by comparing their definitions.
 - ii. Classification : Checking whether an object belongs to a category.
 - iii. Consistency : It is used for category definition to check if membership criteria are logically satisfiable.
4. Generally a problem instance is solved by describing it and then asking if it is subsumed by one of several possible solution categories.
5. The statements in description logic do not contain CLASSIC saying "Bachelors are unmarried adult males" is,
Bachelor = And (Unmarried, Adult, Male).

Q.43 What is non-monotonic reasoning ? What are its types ?

Ans. : Non-Monotonic reasoning is a generic name to a class or a specific **theory of reasoning**. Non-Monotonic reasoning attempts to formalize reasoning with incomplete information by classical logic systems.

Human reasoning is non-monotonic :

People arrive at conclusions only tentatively, based on partial or incomplete information, reserve the right to retract those conclusions, while they learn new facts. Such reasoning is non-monotonic, precisely because the set of accepted conclusions becomes smaller when the set of premises is expanded.

Mathematical reasoning is monotonic :

"If a conclusion follows from given premises A,B,C,... then it also follows from any larger set of premises, as long as the original premises A,B,C,... are included".

The non-monotonic reasoning are of the type :

1. Default Reasoning
2. Circumscription
3. Truth Maintenance Systems.

Default Reasoning

1. This is very common form of non-monotonic reasoning. The conclusions are drawn based on what is most likely to be true.
2. There are two approaches, both are logic type, to Default reasoning, one is Non-monotonic logic and the other is Default logic.

A. Non-Monotonic logic :

- i) It has already been defined. It says, "the truth of a proposition may change when new information (axioms) are added and a logic may be build to allow the statement to be retracted."
- ii) Non-monotonic logic is predicate logic with one extension called **Modal operator M** which means "consistent with everything we know". The purpose of M is to allow consistency.

iii) A way to define consistency with PROLOG notation is :

To show that fact P is true, we attempt to prove $\neg P$.

If we fail we may say that P is consistent since $\neg P$ is false.

For example :

$\forall x \text{ plays_instrument}(x) \wedge \text{Manage}(x) \rightarrow \text{jazz_musician}(x)$

States that for all x, the x plays an instrument and if the fact that x can manage is consistent with all other knowledge then we can conclude that x is a jazz musician.

B. Default logic :

- 1) Default logic initiates a new inference rule :

$$\frac{A:B}{C} \text{ where}$$

A is known as the prerequisite.

B is the justification, and

C as the consequent.

- 2) Read the above inference rule as :

"If A, and if it is consistent with the rest of what is known to assume that B. Then conclude that C".

- 3) The rule says that given the prerequisite, the consequent can be inferred, provided it is consistent with the rest of the data.

- 4) Example : Rule that "birds typically fly" would be represented as,

$$\frac{\text{bird}(x) : \text{flies}(x)}{\text{flies}(x)}$$
 which says

"If x is a bird and the claim that x flies is consistent with what we know, then infer that x flies".

- 5) Now since, all we know about Tweety is that,
 Tweety is a bird, we therefore inferred that "Tweety flies."
- 6) The idea behind non-monotonic reasoning is to reason with first order logic, and if an inference can not be obtained then use the set of default rules available within the first order formulation.
- 7) Applying Default Rules : While applying default rules, it is necessary to check their justifications for consistency, not only with initial data, but also with the consequents of any other default rules that may be applied. The application of one rule may thus block the application of another. To solve this problem, the concept of default theory was extended.
- 8) Default Theory : It consists of a set of premises W and a set of default rules D. An extension for a default theory is a set of sentences E which can be derived from W by applying as many rules of D as possible (together with the rules of deductive inference) without generating inconsistency.
- 9) Note : D the set of default rules has a unique syntax of the form,

$$\frac{\alpha(\bar{x}) : E \beta(\bar{x})}{\gamma(\bar{x})} \text{ where}$$

$\alpha(\bar{x})$ is the prerequisite of the default rule.

$E\beta(\bar{x})$ is the consistency test of the default rule.

$\gamma(\bar{x})$ is the consequent of the default rule.

This rule can be read as for all individual $x_1, \dots, x_m$, if $\alpha(\bar{x})$ is believed and if each of $\beta(\bar{x})$ is consistent with our beliefs, then $\gamma(\bar{x})$ may be believed.

- 10) Example : A Default rule says, "Typically an Indian adult owns a bike".

$$\frac{\text{Indian}(x) \wedge \text{adult}(x) : \exists y \text{ bike}(y) \wedge \text{owns}(x, y)}{\exists y \text{ bike}(y) \wedge \text{owns}(x, y)}$$

The rule is explained below :

- i) The rule is only accessed if we wish to know whether or not 'Ram owns a bike' then an answer can not be deduced from our current beliefs.
- ii) This default rule is applicable, if we can prove from our beliefs that 'Ram is an Indian and an adult', and believing that 'there is some bike that is owned by Ram' does not lead to an inconsistency.
- iii) If these two sets of premises are satisfied, then the rule states that we can conclude that 'Ram owns a bike.'

Circumscription

Circumscription is a non-monotonic logic to formalize the common sense assumption. Circumscription is a formalized rule of conjecture (guess) that can be used along with the rules of inference of first order logic.

Circumscription involves formulating rules of thumb with "abnormality" predicates and then restricting the extension of these predicates, circumscribing them, so that they apply to only those things to which they are currently known.

For Example : Take the case of Bird Tweety.

The rule of thumb is that "birds typically fly" is conditional. The predicate "Abnormal" signifies abnormality with respect to flying ability.

Observe that the rule $\forall x (\text{Bird}(x) \wedge \neg \text{Abnormal}(x) \rightarrow \text{flies})$ does not allow us to infer that "Tweety flies", since we do not know that this is abnormal with respect to flying ability.

But if we add axioms which circumscribe the abnormality predicate to which they are currently known say "Bird Tweety" then the inference can be drawn. This inference is non-monotonic.

Truth Maintenance Systems

1. Reasoning Maintenance Systems (RMS) is a critical part of a reasoning system. Its purpose is to assure that inferences made by the Reasoning System (RS) are valid.
2. The Reasoning system provides the RMS with information about each inference it performs, and in return the RMS provides the RS with information about the whole set of inferences.

3. Several implementations of RMS have been proposed for non-monotonic reasoning. The important ones are the : Truth Maintenance System (TMS) and Assumption-based Truth Maintenance System (ATMS).
4. The TMS maintains the consistency of a knowledge base as soon as new knowledge is added. It considers only one state at a time so it is not possible to manipulate environment.
5. The ATMS is intended to maintain multiple environments.

Properties of Truth Maintenance System (TMS) :

A truth maintenance system maintains consistency in knowledge representation of a knowledge base.

The functions of TMS are :

- 1) Provide justifications for conclusions : When a problem solving system gives an answer to a user's query, an explanation of that answer is required.
Example : An advice to a stockbroker is supported by an explanation of the reasons for that advice. This is constructed by the Inference Engine (IE) by tracing the justification of the assertion.

- 2) Recognize inconsistencies : The Inference Engine (IE) may tell the TMS that some sentences are contradictory. Then, TMS may find that all those sentences are believed true, and report to the IE which can eliminate the inconsistencies by determining the assumptions used and changing them appropriately.

Example : A statement that either Duryodhan, or Shakuni, or Dushasan is guilty together with other statements that Duryodhan is not guilty, Shakuni is not guilty, and Dushasan is not guilty, form a contradiction.

- 3) Support default reasoning : In the absence of any firm knowledge, in many situations we want to reason from default assumptions.

Example : If "Tweety is a bird", then until told otherwise, assume that "Tweety flies" and for justification use the fact that "Tweety is a bird" and the assumptions that "birds fly".

END... ✍